DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Concurrent Banking

CMSC 125 Lab 3

Prepared by: Rene Andre B. Jocsing

Published: March 25, 2026 **Deadline:** May 21, 2026

This laboratory assignment focuses on concurrency control in a multi-threaded banking system using real POSIX threads (pthreads) and synchronization primitives. You will build an in-memory bank that processes concurrent transactions, implements proper isolation through locking, manages a bounded buffer pool with semaphores, and handles deadlock through either prevention or detection. Through this exercise, you will gain hands-on experience with the classical concurrency problems that real systems confront every day.

Background

Modern banking systems must handle thousands of concurrent transactions—deposits, withdrawals, and transfers that read and modify shared account balances. Without proper concurrency control, chaos ensues: one transaction's partial updates become visible to others, two transactions overwrite each other's changes (the lost update problem), or the system deadlocks entirely when transactions wait circularly for locks.

Your task is to build a working multi-threaded banking system that correctly serializes conflicting operations while allowing safe concurrency where possible. This is not a simulation—you will use real pthreads, real mutexes and reader-writer locks, real semaphores, and experience real race conditions if you get the synchronization wrong.

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Implement multi-threaded programs using the POSIX threads API (`pthread_create`, `pthread_join`)
- Protect shared data structures with mutual exclusion using `pthread_mutex_t`
- Implement reader-writer synchronization using `pthread_rwlock_t`
- Solve the bounded-buffer problem using semaphores (`sem_t`)
- Implement deadlock prevention through lock ordering OR deadlock detection through cycle detection
- Simulate time progression using a timer thread
- Debug race conditions and data corruption using ThreadSanitizer
- Measure concurrency performance (throughput, lock contention, wait times)
- Explain how classical concurrency problems appear in real banking systems

---

Task

Build a concurrent banking system (`bankdb`) that:

- Stores bank accounts with balances (integer centavos)
- Executes transactions concurrently using real pthreads
- Uses a timer thread to simulate time progression (global clock)
- Implements transaction isolation using reader-writer locks
- Prevents OR detects deadlock (choose one strategy)
- Manages a bounded buffer pool with semaphores
- Reads transaction workloads from provided trace files
- Outputs transaction logs, lock wait times, and concurrency metrics
- Demonstrates correctness under ThreadSanitizer with zero warnings

---

System Architecture

Data Model

Everything in this system is money. The database contains bank accounts with integer balances in centavos:

```c
#define MAX_ACCOUNTS 100

typedef struct {
    int account_id;          // Account number
    int balance_centavos;    // Balance in centavos
    pthread_rwlock_t lock;   // Per-account lock
} Account;

typedef struct {
    Account accounts[MAX_ACCOUNTS];
    int num_accounts;
    pthread_mutex_t bank_lock;  // Protects bank metadata
} Bank;
```

Transaction Types

A transaction is a sequence of banking operations:

```c
typedef enum {
    OP_DEPOSIT,   // Add money to account
    OP_WITHDRAW,  // Remove money from account
    OP_TRANSFER,  // Move money between two accounts
    OP_BALANCE,   // Read account balance
} OpType;

typedef struct {
    OpType type;
    int account_id;          // Primary account
    int amount_centavos;     // Amount in centavos
    int target_account;      // For TRANSFER only
} Operation;

typedef enum {
    TX_RUNNING,
    TX_COMMITTED,
    TX_ABORTED
```

```
} TxStatus;

typedef struct {
    int tx_id;
    Operation ops[256];    // Max 256 operations per transaction
    int num_ops;
    int start_tick;       // When transaction should start
    pthread_t thread;

    // Timing (measured in ticks)
    int actual_start;
    int actual_end;
    int wait_ticks;

    // Status
    TxStatus status;
} Transaction;
```

Time Simulation

Your system must simulate time using a **timer thread** that increments a global clock:

```
// Global simulation clock (shared by all threads)
volatile int global_tick = 0;
pthread_mutex_t tick_lock;
pthread_cond_t tick_changed;

// Timer thread increments clock every TICK_INTERVAL_MS
void* timer_thread(void* arg) {
    while (simulation_running) {
        pthread_mutex_lock(&tick_lock);
        usleep(TICK_INTERVAL_MS * 1000);  // Sleep to simulate a tick
        global_tick++;
        pthread_cond_broadcast(&tick_changed);  // Wake waiting
        pthread_mutex_unlock(&tick_lock);
    }
    return NULL;
}

// Transactions wait until their start_tick
void wait_until_tick(int target_tick) {
    pthread_mutex_lock(&tick_lock);
    while (global_tick < target_tick) {
        pthread_cond_wait(&tick_changed, &tick_lock);
    }
    pthread_mutex_unlock(&tick_lock);
}
```

Think: Why do we need a timer thread instead of just processing operations sequentially? What concurrency does this enable?

Required Features

Part 1: Multi-threaded Transaction Execution

Each transaction runs in its own pthread and waits for the correct tick before starting:

```c
void* execute_transaction(void* arg) {
    Transaction* tx = (Transaction*)arg;

    // Wait until scheduled start time
    wait_until_tick(tx->start_tick);

    tx->actual_start = global_tick;

    for (int i = 0; i < tx->num_ops; i++) {
        Operation* op = &tx->ops[i];

        int tick_before = global_tick;

        switch (op->type) {
            case OP_DEPOSIT:
                deposit(op->account_id, op->amount_centavos);
                break;

            case OP_WITHDRAW:
                if (!withdraw(op->account_id, op->amount_centavos)) {
                    // Insufficient funds - abort transaction
                    tx->status = TX_ABORTED;
                    return NULL;
                }
                break;

            case OP_TRANSFER:
                if (!transfer(op->account_id, op->target_account,
                              op->amount_centavos)) {
                    tx->status = TX_ABORTED;
                    return NULL;
                }
                break;

            case OP_BALANCE:
                int balance = get_balance(op->account_id);
                printf("T%d: Account %d balance = PHP %d.%02d\n",
                       tx->tx_id, op->account_id,
                       balance / 100, balance % 100);
                break;
        }

        tx->wait_ticks += (global_tick - tick_before);
    }

    tx->actual_end = global_tick;
    tx->status = TX_COMMITTED;
    return NULL;
}
```

Part 2: Reader-Writer Locks for Account Access

Use `pthread_rwlock_t` to allow multiple balance queries or one writer per account:

```c
int get_balance(int account_id) {
    Account* acc = &bank.accounts[account_id];

    pthread_rwlock_rdlock(&acc->lock);
    int balance = acc->balance_centavos;
```

```
    pthread_rwlock_unlock(&acc->lock);

    return balance;
}

void deposit(int account_id, int amount_centavos) {
    Account* acc = &bank.accounts[account_id];

    pthread_rwlock_wrlock(&acc->lock);
    acc->balance_centavos += amount_centavos;
    pthread_rwlock_unlock(&acc->lock);
}

bool withdraw(int account_id, int amount_centavos) {
    Account* acc = &bank.accounts[account_id];

    pthread_rwlock_wrlock(&acc->lock);

    if (acc->balance_centavos < amount_centavos) {
        pthread_rwlock_unlock(&acc->lock);
        return false;  // Insufficient funds
    }

    acc->balance_centavos -= amount_centavos;
    pthread_rwlock_unlock(&acc->lock);
    return true;
}

bool transfer(int from_id, int to_id, int amount_centavos) {
    // This is where deadlock can occur!
    // See Part 3 for proper implementation
}
```

Part 3: Deadlock Handling (Choose ONE Strategy)

The `transfer` operation acquires locks on two accounts. If two transactions transfer in opposite directions simultaneously, deadlock occurs.

You must choose and implement ONE of the following strategies:

Strategy A: Deadlock Prevention via Lock Ordering

Always acquire locks in ascending order of account ID:

```
bool transfer(int from_id, int to_id, int amount_centavos) {
    // Acquire locks in consistent order to prevent deadlock
    int first = (from_id < to_id) ? from_id : to_id;
    int second = (from_id < to_id) ? to_id : from_id;

    Account* acc_first = &bank.accounts[first];
    Account* acc_second = &bank.accounts[second];

    pthread_rwlock_wrlock(&acc_first->lock);
    pthread_rwlock_wrlock(&acc_second->lock);

    // Check sufficient funds
    Account* from_acc = &bank.accounts[from_id];
    if (from_acc->balance_centavos < amount_centavos) {
        pthread_rwlock_unlock(&acc_second->lock);
```

```
        pthread_rwlock_unlock(&acc_first->lock);
        return false;
    }

    // Perform transfer
    bank.accounts[from_id].balance_centavos -= amount_centavos;
    bank.accounts[to_id].balance_centavos += amount_centavos;

    pthread_rwlock_unlock(&acc_second->lock);
    pthread_rwlock_unlock(&acc_first->lock);
    return true;
}
```

Which Coffman condition does lock ordering break?

Strategy B: Deadlock Detection via Wait-For Graph

Maintain a wait-for graph and detect cycles using DFS:

```
typedef struct {
    int tx_id;
    int waiting_for_tx;  // -1 if not waiting
    int waiting_for_account;
} WaitForEntry;

WaitForEntry wait_graph[MAX_TRANSACTIONS];
pthread_mutex_t graph_lock;

// When a transaction blocks on a lock, record it
void record_wait(int tx_id, int account_id, int holder_tx) {
    pthread_mutex_lock(&graph_lock);
    wait_graph[tx_id].tx_id = tx_id;
    wait_graph[tx_id].waiting_for_tx = holder_tx;
    wait_graph[tx_id].waiting_for_account = account_id;
    pthread_mutex_unlock(&graph_lock);
}

// DFS-based cycle detection
bool has_cycle(int tx_id, bool* visited, bool* rec_stack) {
    visited[tx_id] = true;
    rec_stack[tx_id] = true;

    int next = wait_graph[tx_id].waiting_for_tx;
    if (next != -1) {
        if (!visited[next]) {
            if (has_cycle(next, visited, rec_stack)) {
                return true;
            }
        } else if (rec_stack[next]) {
            return true;  // Cycle detected!
        }
    }

    rec_stack[tx_id] = false;
    return false;
}

bool detect_deadlock() {
    pthread_mutex_lock(&graph_lock);
```

```
41
42      bool visited[MAX_TRANSACTIONS] = {false};
43      bool rec_stack[MAX_TRANSACTIONS] = {false};
44
45      for (int i = 0; i < num_active_transactions; i++) {
46          if (!visited[i]) {
47              if (has_cycle(i, visited, rec_stack)) {
48                  pthread_mutex_unlock(&graph_lock);
49                  return true;
50              }
51          }
52      }
53
54      pthread_mutex_unlock(&graph_lock);
55      return false;
56  }
```

If deadlock is detected, abort the youngest transaction in the cycle.

Note: Strategy A (prevention) is easier to implement correctly. Strategy B (detection) is more challenging but demonstrates deeper understanding of deadlock theory.

Part 4: Bounded Buffer Pool with Semaphores

Purpose: This component demonstrates the bounded-buffer (producer-consumer) problem. The buffer pool has a fixed size, and semaphores coordinate producers (transactions loading accounts) and consumers (transactions unloading accounts).

The bank maintains a limited buffer pool for loading account data from the “disk” (simulated):

```
1   #define BUFFER_POOL_SIZE 5
2
3   typedef struct {
4       int account_id;
5       Account* data;
6       bool in_use;
7   } BufferSlot;
8
9   typedef struct {
10      BufferSlot slots[BUFFER_POOL_SIZE];
11      sem_t empty_slots;
12      sem_t full_slots;
13      pthread_mutex_t pool_lock;
14  } BufferPool;
15
16  void init_buffer_pool(BufferPool* pool) {
17      sem_init(&pool->empty_slots, 0, BUFFER_POOL_SIZE);
18      sem_init(&pool->full_slots, 0, 0);
19      pthread_mutex_init(&pool->pool_lock, NULL);
20  }
21
22  // Load account into buffer pool (producer)
23  void load_account(BufferPool* pool, int account_id) {
24      sem_wait(&pool->empty_slots);  // Wait for empty slot
25
26      pthread_mutex_lock(&pool->pool_lock);
27
28      // Find empty slot and load account
29      for (int i = 0; i < BUFFER_POOL_SIZE; i++) {
30          if (!pool->slots[i].in_use) {
31              pool->slots[i].account_id = account_id;
```

```
            pool->slots[i].data = &bank.accounts[account_id];
            pool->slots[i].in_use = true;
            break;
        }
    }

    pthread_mutex_unlock(&pool->pool_lock);

    sem_post(&pool->full_slots);  // Signal slot is full
}

// Unload account from buffer pool (consumer)
void unload_account(BufferPool* pool, int account_id) {
    sem_wait(&pool->full_slots);  // Wait for full slot

    pthread_mutex_lock(&pool->pool_lock);

    // Find and unload account
    for (int i = 0; i < BUFFER_POOL_SIZE; i++) {
        if (pool->slots[i].in_use &&
            pool->slots[i].account_id == account_id) {
            pool->slots[i].in_use = false;
            pool->slots[i].account_id = -1;
            break;
        }
    }

    pthread_mutex_unlock(&pool->pool_lock);

    sem_post(&pool->empty_slots);  // Signal slot is empty
}
```

Integration with banking operations: You must decide when to load/unload accounts to/from the buffer pool. This decision must be documented in your `design.md`. Possible strategies:

- Load on first access, unload on transaction commit
- Load all accounts transaction needs at start, unload all at end
- Load/unload per operation
- Implement an LRU eviction policy

There is no single correct answer—justify your choice with reasoning about performance and correctness.

Input Format

Trace File Format

We provide trace files in the following format:

```
# Banking transaction trace
# Format: TxID  StartTick  Operation  AccountID  [Amount]  [TargetAccount]

# Transaction 1: Simple deposit
T1  0  DEPOSIT   10  5000

# Transaction 2: Withdrawal
T2  1  WITHDRAW  10  2000

```

```
# Transaction 3: Transfer (can deadlock with T4)
T3  2  TRANSFER  10  20  3000

# Transaction 4: Concurrent transfer in opposite direction
T4  2  TRANSFER  20  10  1500

# Transaction 5: Balance inquiry
T5  5  BALANCE   10
```

Lines beginning with # are comments. Operations are:

- `DEPOSIT account_id amount`: Add amount centavos to account
- `WITHDRAW account_id amount`: Remove amount centavos (abort if insufficient)
- `TRANSFER from_id to_id amount`: Move amount from one account to another
- `BALANCE account_id`: Read and print account balance

Initial Account Balances

Your program must load initial balances from a separate file:

```
# AccountID  InitialBalanceCentavos
0   10000
1   25000
10  50000
20  30000
```

Command-line Interface

```
$ ./bankdb --accounts=accounts.txt --trace=trace.txt \
           --deadlock=prevention --tick-ms=100

$ ./bankdb --accounts=accounts.txt --trace=trace.txt \
           --deadlock=detection --tick-ms=50 --verbose
```

Required options:

- `--accounts=FILE`: Initial account balances
- `--trace=FILE`: Transaction workload
- `--deadlock=prevention|detection`: Deadlock strategy
- `--tick-ms=N`: Milliseconds per tick (default: 100)
- `--verbose`: Print detailed operation logs

Expected Output

Transaction Log

```
=== Banking System Execution Log ===
Timer thread started (tick interval: 100ms)

Tick 0:
  T1 started: DEPOSIT account 10 amount PHP 50.00

Tick 1:
  T1 completed: DEPOSIT successful
```

```
  T2 started: WITHDRAW account 10 amount PHP 20.00

Tick 2:
  T2 completed: WITHDRAW successful
  T3 started: TRANSFER from 10 to 20 amount PHP 30.00
  T4 started: TRANSFER from 20 to 10 amount PHP 15.00

Tick 3:
  T3 acquired lock on account 10
  T4 acquired lock on account 20
  [DEADLOCK PREVENTED] Lock ordering: T3 waiting for account 20
  [DEADLOCK PREVENTED] Lock ordering: T4 waiting for account 10

Tick 4:
  T3 completed: TRANSFER successful
  T4 completed: TRANSFER successful

Tick 5:
  T5 started: BALANCE account 10
  T5: Account 10 balance = PHP 145.00

=== Summary ===
Total transactions: 5
Committed: 5
Aborted: 0
Total ticks: 6
ThreadSanitizer warnings: 0
```

Detailed Metrics

```
=== Transaction Performance Metrics ===
TxID | StartTick | ActualStart | End | WaitTicks | Status
-----|-----------|-------------|-----|-----------|----------
T1   |     0     |      0      |  1  |     0     | COMMITTED
T2   |     1     |      1      |  2  |     0     | COMMITTED
T3   |     2     |      2      |  4  |     1     | COMMITTED
T4   |     2     |      2      |  4  |     1     | COMMITTED
T5   |     5     |      5      |  5  |     0     | COMMITTED

Average wait time: 0.4 ticks
Throughput: 5 transactions / 6 ticks = 0.83 tx/tick
```

Buffer Pool Statistics

```
=== Buffer Pool Report ===
Pool size: 5 slots
Total loads: 8
Total unloads: 8
Peak usage: 4 slots
Blocked operations (pool full): 0
```

Provided Test Cases

We provide the following trace files that your program MUST handle correctly.

Test 1: No Conflicts (trace_simple.txt)

```
# Single-threaded sequential operations
T1  0  DEPOSIT   10  10000
T1  1  WITHDRAW  10  2000
T1  2  BALANCE   10
```

Expected: All operations succeed, account 10 final balance = PHP 80.00

Test 2: Concurrent Readers (trace_readers.txt)

```
# Multiple transactions reading same account simultaneously
T1  0  BALANCE   10
T2  0  BALANCE   10
T3  0  BALANCE   10
T4  0  BALANCE   10
```

Expected: All complete at same tick (reader-writer lock allows concurrent reads)

Test 3: Deadlock Scenario (trace_deadlock.txt)

```
# Two transfers in opposite directions
T1  0  TRANSFER  10  20  5000
T2  0  TRANSFER  20  10  3000
```

Expected:

- With `--deadlock=prevention`: Both succeed, lock ordering prevents deadlock
- With `--deadlock=detection`: Deadlock detected, one transaction aborted

Test 4: Insufficient Funds (trace_abort.txt)

```
# Initial: Account 10 has PHP 100.00
T1  0  WITHDRAW  10  15000
```

Expected: T1 aborts with insufficient funds error

Test 5: Buffer Pool Saturation (trace_buffer.txt)

```
# Load more accounts than buffer pool size (pool = 5 slots)
T1  0  DEPOSIT   1  1000
T2  0  DEPOSIT   2  1000
T3  0  DEPOSIT   3  1000
T4  0  DEPOSIT   4  1000
T5  0  DEPOSIT   5  1000
T6  0  DEPOSIT   6  1000
```

Expected: T6 blocks until a buffer slot is freed (demonstrates bounded buffer)

Testing Strategy

Correctness Testing

Test 1: ThreadSanitizer (Zero Warnings Required)

```
$ make debug  # Compiles with -fsanitize=thread
$ ./bankdb --accounts=accounts.txt --trace=trace_readers.txt \
           --deadlock=prevention

# YOUR PROGRAM MUST PRODUCE ZERO ThreadSanitizer WARNINGS
# Any data race detected = automatic failure
```

Test 2: Deadlock Handling

```
# Prevention strategy
$ ./bankdb --accounts=accounts.txt --trace=trace_deadlock.txt \
           --deadlock=prevention --verbose

# Expected output must show:
# - Lock ordering applied
# - Both transactions complete
# - No deadlock occurred

# Detection strategy
$ ./bankdb --accounts=accounts.txt --trace=trace_deadlock.txt \
           --deadlock=detection --verbose

# Expected output must show:
# - Deadlock detected
# - One transaction aborted
# - Other transaction completed
```

Test 3: Balance Consistency

After all transactions complete, verify total money in system is conserved:

```
# Your program must print:
Initial total: PHP 1150.00
Final total:   PHP 1150.00
Conservation check: PASSED
```

Performance Testing

Compare reader-writer locks vs. plain mutexes on read-heavy workload:

```
# Modify your code to use pthread_mutex_t instead of pthread_rwlock_t
$ ./bankdb --trace=trace_readers.txt --tick-ms=10
# Record: "Completed in X ticks"

# Switch back to pthread_rwlock_t
$ ./bankdb --trace=trace_readers.txt --tick-ms=10
# Record: "Completed in Y ticks"

# rwlock should be faster (lower Y) on read-heavy workload
```

Document this comparison in your `design.md`.

Implementation Notes

Timing Measurements

Use the global tick counter, not wall-clock time:

```
void deposit(int account_id, int amount_centavos) {
    int tick_before = global_tick;

    pthread_rwlock_wrlock(&bank.accounts[account_id].lock);

    int tick_after = global_tick;
    int wait_ticks = tick_after - tick_before;

    // Perform operation
    bank.accounts[account_id].balance_centavos += amount_centavos;

    pthread_rwlock_unlock(&bank.accounts[account_id].lock);
}
```

Thread Synchronization for Timer

The timer thread must signal waiting transactions when the tick advances:

```
void* timer_thread(void* arg) {
    while (!all_transactions_done) {
        usleep(tick_interval_ms * 1000);

        pthread_mutex_lock(&tick_lock);
        global_tick++;

        // Wake all transactions waiting for this tick
        pthread_cond_broadcast(&tick_changed);

        pthread_mutex_unlock(&tick_lock);
    }
    return NULL;
}
```

Common Pitfalls

- **Forgetting to unlock**: Always release locks even on error paths
- **Lock after free**: Release lock AFTER you're done reading/writing the data
- **Deadlock without ordering**: If you chose prevention, you MUST acquire locks in consistent order
- **Race on global_tick**: Always read `global_tick` while holding `tick_lock`
- **Semaphore initialization**: `sem_init(&sem, 0, count)` — second arg is 0 for threads (not processes)
- **Money conservation**: Sum of all balances in the system must remain constant

---

Proposed Project Structure

```
bankdb/
|-- Makefile
|-- README.md
|-- include/
|   |-- bank.h           # Bank and account structures
|   |-- transaction.h    # Transaction and operation types
|   |-- timer.h          # Timer thread and clock functions
|   |-- lock_mgr.h       # Lock ordering or deadlock detection
|   |-- buffer_pool.h    # Buffer pool with semaphores
|   +-- metrics.h        # Statistics collection
|-- src/
|   |-- main.c           # CLI parsing, initialization
|   |-- bank.c           # Account operations
|   |-- transaction.c    # Transaction execution thread
|   |-- timer.c          # Timer thread implementation
|   |-- lock_mgr.c       # Deadlock prevention or detection
|   |-- buffer_pool.c    # Bounded buffer implementation
|   |-- metrics.c        # Metrics calculation and reporting
|   +-- utils.c          # Parsing, error handling
|-- tests/
|   |-- accounts.txt           # Initial account balances
|   |-- trace_simple.txt       # Test 1
|   |-- trace_readers.txt      # Test 2
|   |-- trace_deadlock.txt     # Test 3
|   |-- trace_abort.txt        # Test 4
|   +-- trace_buffer.txt       # Test 5
+-- docs/
    +-- design.md              # Design justifications
```

Design Documentation

Your `docs/design.md` must address the following questions.

Required Discussion Topics

1. **Deadlock Strategy Choice**
   - Which strategy did you choose (prevention or detection)?
   - Why did you choose this strategy?
   - If prevention: Prove that lock ordering eliminates circular wait. Which Coffman condition is broken?
   - If detection: Explain your cycle detection algorithm. How do you choose which transaction to abort?
2. **Buffer Pool Integration**
   - When do you load accounts into the buffer pool?
   - When do you unload them?
   - What happens if the pool is full when a transaction needs an account?
   - Justify your design with reasoning about performance and correctness
3. **Reader-Writer Lock Performance**
   - Show benchmark results comparing `pthread_mutex_t` vs `pthread_rwlock_t`
   - On which workload (trace file) does rwlock show the biggest improvement?
   - Why does rwlock help on read-heavy workloads?
4. **Timer Thread Design**
   - Why is a separate timer thread necessary?
   - What would break if you removed the timer and processed operations sequentially?

- How does the timer thread enable true concurrency testing?

Deliverables

Your pair's GitHub repository must contain:

1. All source files (`.c` and `.h`) with proper documentation
2. `Makefile` with targets:
   - `all`: Compile with `-pthread -O2 -Wall -Wextra`
   - `debug`: Compile with `-g -fsanitize=thread -pthread`
   - `clean`: Remove binaries and object files
   - `test`: Run all 5 provided test cases
3. `README.md` with:
   - Complete names of both group members
   - Compilation and usage instructions
   - List of implemented features
   - Known limitations (if any)
4. `docs/design.md` addressing all 4 required discussion topics
5. Screenshots or logs demonstrating:
   - ThreadSanitizer producing zero warnings on all test cases
   - Deadlock handling (prevention or detection) working correctly
   - Buffer pool blocking when full, then unblocking
   - Balance conservation check passing

To submit, invite the instructor as a collaborator to your repository. Commits after grading will not be considered.

Then, submit the following **individually** via email:

1. `reflection.txt`: Which concurrency problem (mutual exclusion, reader-writer, bounded buffer, deadlock) was most difficult to implement correctly and why?
2. `peer.txt`: Assessment of your partner's contributions and collaboration.
3. GitHub repository link (for verification)

Subject line: `[CMSC 125 Lab] Lab 3: Surname, Initials`

Example: `[CMSC 125 Lab] Lab 3: Sanchez, SM`

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

Important Dates

Progress reports and laboratory defense may be booked only during the dates and hours defined in the syllabus. It is mandatory to book appointments ahead of time on the course's booking page (also accessible on the course site).

Activity	Monday	Wednesday	Thursday
Week 1 Progress Report	Apr 6	Apr 8	Apr 9
Week 2 Progress Report	Apr 13	Apr 15	Apr 16
Week 3 Progress Report	Apr 20	Apr 22	Apr 23
Week 4 Laboratory Defense	**Apr 27**	**Apr 29**	**Apr 30**

The instructor must verify appointments ahead of time before they can be considered valid. See the syllabus for proper hours and more details.

Grading Rubric

Criteria	Excellent (90–100%)	Good (75–89%)	Fair (60–74%)	Poor (0–59%)
System Architecture (25%)	Modular design with clean separation of concerns; robust data structures; efficient resource usage.	Mostly modular; logic is functional but slightly coupled; data structures are appropriate.	Significant logic sprawl; monolithic functions; inconsistent data handling or hard-coded limits.	Spaghetti code; no modularity; violates basic systems programming principles.
Robustness (20%)	Handles complex edge cases and race conditions; perfect resource lifecycle; graceful error recovery.	Handles core features well; minor issues with fringe edge cases or synchronization logic.	Basic features work, but system is unstable; frequent resource leaks or intermittent errors.	Fails core logic; program crashes on unexpected input; incorrect usage of fundamental syscalls.
Code Engineering (10%)	No memory leaks; all syscalls check return codes; uses `perror` appropriately; safe pointer usage.	Minor leaks or missing checks on non-critical syscalls; mostly safe pointer arithmetic.	Inconsistent error handling; frequent unsafe operations; significant leaks or lack of bounds checking.	Frequent segmentation faults; silent failures of syscalls; no evidence of memory management.
Collaboration (20%)	Professional Git usage: atomic, semantic commits; use of feature branches; evidence of team collaboration.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history or shows no evidence of teamwork.
Technical Defense (25%)	Both members articulate the low-level mechanics; handles what-if scenarios and code-tracing confidently.	Clear architectural explanation; both members participate meaningfully; logic delivery is sound.	Unclear explanations of system mechanics; uneven participation; struggles with logic-flow questions.	Unprepared; cannot explain system flow or syscall interactions; unable to defend design decisions.